

# Maubach Subdivision on $\mathbb{CP}^2$

Vinicius Mello

September 27, 2026

## Abstract

Maubach’s edge-based star-subdivision scheme, as developed in Chapter 4 of the author’s thesis, is a purely combinatorial criterion (the “Maubach invariants”) that guarantees a longest-edge-style bisection algorithm propagates correctly across an entire mesh. The thesis proves this criterion is satisfied by two specific constructions: the canonical (Coxeter–Freudenthal–Kuhn) triangulation of a product of simplices, and the barycentric subdivision of an arbitrary mesh. Both routes rely on a *global* vertex ranking that is not available in general. This report verifies the thesis’s Lemmas 5–7 (finding and correcting a symbol lost in the source PDF’s text layer along the way), and then shows, with a working implementation, that the same criterion extends well beyond those two cases: to meshes in dimension greater than the classical  $\leq 3$  setting, and to a triangulation with no group-orbit or flag structure at all, verified compatible purely by an offline combinatorial search. The application used throughout is the adaptive triangulation of a smooth algebraic curve  $F(X, Y, Z) = 0$  – a Riemann surface – realized as a codimension-2 locus inside the complex projective plane  $\mathbb{CP}^2$  (a real 4-manifold), triangulated by Gaifullin’s 108-cell complex. We describe how this mesh satisfies Maubach compatibility, how an economical, pointerless cell representation (Atalay & Mount’s LPT codes) is adapted to a seed with no algebraic symmetry, and the final version of the resulting adaptive triangulation algorithm, based on continuation rather than global refinement.

## Contents

|          |                                                                          |          |
|----------|--------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                      | <b>2</b> |
| <b>2</b> | <b>Star subdivisions and the Maubach invariant</b>                       | <b>3</b> |
| 2.1      | The Maubach invariants . . . . .                                         | 3        |
| 2.2      | Lemma 5: propagation direction, verified . . . . .                       | 3        |
| 2.3      | Lemma 6: the child relation . . . . .                                    | 4        |
| 2.4      | Lemma 7: propagation preserves the invariants . . . . .                  | 4        |
| 2.5      | Two constructive routes, and their common mechanism . . . . .            | 5        |
| <b>3</b> | <b>Compatibility beyond the cube: a combinatorial search</b>             | <b>6</b> |
| 3.1      | Definition 9 as a coloring problem . . . . .                             | 6        |
| 3.2      | Gaifullin’s triangulation . . . . .                                      | 6        |
| 3.3      | A cautionary counterexample: Kühnel’s triangulation . . . . .            | 7        |
| <b>4</b> | <b>An economical mesh representation: LPT codes and neighbor finding</b> | <b>7</b> |
| 4.1      | Atalay & Mount’s pointerless codes . . . . .                             | 7        |
| 4.2      | glpt: extending LPT to a non-orbit seed . . . . .                        | 8        |
| 4.3      | The neighbor function . . . . .                                          | 8        |
| 4.4      | The current-leaf set: glpt_tree . . . . .                                | 9        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>5</b> | <b>A continuation algorithm for Riemann-surface triangulation</b>  | <b>10</b> |
| 5.1      | Problem setup . . . . .                                            | 10        |
| 5.2      | From global refinement to continuation . . . . .                   | 10        |
| 5.3      | The algorithm . . . . .                                            | 11        |
| 5.4      | Measured results . . . . .                                         | 11        |
| 5.5      | Limitations . . . . .                                              | 12        |
| <b>6</b> | <b>Conclusion</b>                                                  | <b>13</b> |
| <b>A</b> | <b>Products of already-compatible meshes</b>                       | <b>13</b> |
| <b>B</b> | <b>Translated proofs of Theorems 8–10, and Maubach’s algorithm</b> | <b>15</b> |

## 1 Introduction

Star-subdivision schemes (Chapter 4 of the thesis [1]) formalize a simple idea: a mesh can be refined automatically, guided only by its own combinatorial data (the partial order on vertices, adjacency, and a hierarchy level), without ever consulting geometry. Two such schemes are developed there in parallel – Mitchell’s, based on splitting a distinguished *facet*, and Maubach’s, based on splitting a distinguished *edge* (the classical “longest-edge” bisection of Kuhn’s canonical triangulation of a cube). Both are given as an *invariant*  $\mathcal{I}$  (a combinatorial property a conforming mesh may or may not have) together with a subdivision algorithm that is proved to preserve  $\mathcal{I}$  forever, once it holds.

The thesis proves Maubach’s invariant is satisfied by exactly two constructions: the canonical triangulation of a product of  $n$  simplices (Theorem 9 – of which Kuhn’s triangulation of a cube, a product of  $n$  intervals, is the special case actually used throughout the rest of the thesis) and the barycentric subdivision of an arbitrary mesh (Theorem 10). Both proofs go through the same mechanism: a *global* function that assigns every vertex of the whole complex a rank in  $\{0, \dots, m\}$ , such that every top-dimensional cell sees each rank exactly once. Neither construction says anything about whether an *arbitrary* triangulation – one with no such global ranking built in – can be made Maubach-compatible.

This report’s goal is to show that it can, and that the resulting machinery genuinely generalizes past the setting the thesis develops it in: dimension  $\leq 3$ , and meshes derived from Kuhn’s cube. We do this constructively, through a concrete application developed over several work sessions: the adaptive triangulation of a smooth plane algebraic curve (a Riemann surface)  $F(X, Y, Z) = 0$ , realized not as a pair of identified Riemann spheres but directly as a codimension-2 real submanifold of  $\mathbb{CP}^2$  – a genuinely 4-dimensional real ambient space, triangulated by A. Gaifullin’s 15-vertex, 108-cell minimal-vertex triangulation of  $\mathbb{CP}^2$  [7], a mesh with no product structure and no flag/rank structure inherited by construction.

The report is organized as follows. Section 2 recapitulates the relevant part of Chapter 4 – star subdivisions, the Maubach invariants, and Lemmas 5–7 – verifying each proof directly (Section 2 includes one genuine correction: the PDF’s text layer silently drops the comparison symbols in Lemma 5’s statement; Section 2.2 reconstructs and confirms the correct direction both from the proof and from the original page image). Section 3 shows how Gaifullin’s triangulation is made Maubach-compatible: not via a global rank, but by reformulating Definition 9 as a graph-coloring search on the mesh’s dual graph, solved once, offline, by exact backtracking, and cross-checked against GAP’s [9] `simpcomp` [10] package. Section 4 describes the economical, pointerless cell representation this project built on top of that mesh – an extension of Atalay & Mount’s LPT codes [6] – with particular attention to what changes in the neighbor-finding rules once the seed cells are no longer a single group orbit. Section 5 describes, in full detail, the current (and, as of this writing, final) version of the Riemann-surface triangulation algorithm,

which abandons global refinement in favor of continuation: following the curve outward from a single confirmed contact cell, using exactly the neighbor machinery of Section 4.

## 2 Star subdivisions and the Maubach invariant

We use the thesis's own notation throughout, restated here only as needed. A *star subdivision* of a complex  $K$  at a simplex  $\sigma$ , replacing  $\sigma$ 's star by a cone from a new vertex  $v$  over  $\text{link}(\sigma, K)$  joined with  $\partial\sigma$ , is written  $K' = K(\sigma, v)$ . An *ordered* star subdivision  $K' = K(\sigma, v)_l$  additionally records that  $v$  occupies position  $l$  in every cell of  $\text{st}(v, K')$ . If  $\sigma$  is itself an  $m$ -dimensional cell  $\langle v_0, \dots, v_m \rangle$  and  $\delta = [j_0, \dots, j_n]\sigma$  is one of its faces, then  $K(\delta, v)_l$  produces  $n + 1$  *child cells*  $c_0\sigma, \dots, c_n\sigma$ , one per facet of  $\partial\delta$ , related by

$$\partial_l c_i \sigma = \partial_{j_{n-i}} \sigma.$$

A finite sequence of ordered star subdivisions  $\tau = (\delta_1, v_1)_{l_1} \cdots (\delta_q, v_q)_{l_q}$  imposes a parent/child hierarchy on cells, tracked by a function  $\text{level}(\cdot)$  with  $\text{level}(\sigma) = 0$  for  $\sigma \in K$  and  $\text{level}(c_i\sigma) = \text{level}(\sigma) + 1$ .

A *subdivision scheme* consists of an invariant  $\mathcal{I}$  (referring only to combinatorial data: the vertex order, adjacency, level) and an algorithm  $\text{Subdivide}(K, \sigma)$  that, to subdivide a single cell  $\sigma$ , first propagates a cascade of subdivisions elsewhere in the mesh as needed to keep  $\mathcal{I}$  true throughout, before finally splitting  $\sigma$  itself. The property used constantly below is

$$\partial_i \sigma = \partial_j \omega \implies |\text{level}(\sigma) - \text{level}(\omega)| \leq 1. \quad (\text{OneLevel})$$

### 2.1 The Maubach invariants

Maubach's scheme [2] splits the edge  $[0, \text{type}(\sigma)]$  of a cell, where

$$\text{type}(\sigma) := m - (\text{level}(\sigma) \bmod m) \in \{1, \dots, m\}.$$

**Definition** (Definition 9). *An  $m$ -dimensional mesh  $K$  satisfies the Maubach invariants if, besides (OneLevel), for any two adjacent cells  $\sigma, \omega$  with  $\partial_i \sigma = \partial_j \omega$ ,*

$$\text{level}(\sigma) = \text{level}(\omega) \implies i = j \quad \text{or} \quad \{i, j\} = \{0, \text{type}(\omega)\}, \quad (\text{MaubachA})$$

$$\text{level}(\sigma) = \text{level}(\omega) + 1 \implies i = \text{type}(\omega) \quad \text{and} \quad j \in \{0, \text{type}(\omega)\}. \quad (\text{MaubachB})$$

Writing  $\text{split}(\sigma) := \sigma[0, \text{type}(\sigma)]$ , (MaubachA) says that two same-level adjacent cells either agree exactly on which vertex their shared facet omits, or disagree in exactly the pair  $\{0, \text{type}(\omega)\}$  – which, as the thesis shows immediately after the definition, still forces  $\text{split}(\sigma) = \text{split}(\omega)$ . (MaubachB) says that if  $\omega$  is one level shallower than  $\sigma$ , subdividing  $\text{split}(\omega)$  will hand one of  $\omega$ 's children exactly the facet  $\sigma$  already has.

### 2.2 Lemma 5: propagation direction, verified

**Lemma** (Lemma 5). *Let  $K$  be a mesh satisfying the Maubach invariants and  $\sigma, \omega \in K$  two adjacent cells with  $\partial_i \sigma = \partial_j \omega$ . Then*

$$\begin{cases} \text{level}(\omega) \leq \text{level}(\sigma), & \text{if } i \notin \{0, \text{type}(\sigma)\} \\ \text{level}(\omega) \geq \text{level}(\sigma), & \text{if } i \in \{0, \text{type}(\sigma)\}. \end{cases}$$

*Moreover, if  $\text{level}(\omega) < \text{level}(\sigma)$ , then  $i = \text{type}(\omega)$ .*

*Proof.* If  $\text{level}(\omega) > \text{level}(\sigma)$ , then by (OneLevel)  $\text{level}(\omega) = \text{level}(\sigma) + 1$ . Applying (MaubachB) with the roles of the two cells matched so that  $\omega$  is the deeper one (i.e. with  $\sigma$  playing (MaubachB)’s “ $\omega$ ”), we get  $i \in \{0, \text{type}(\sigma)\}$ , proving the first case (by contraposition:  $i \notin \{0, \text{type}(\sigma)\}$  rules out  $\text{level}(\omega) > \text{level}(\sigma)$ , leaving  $\text{level}(\omega) \leq \text{level}(\sigma)$  by (OneLevel)). If  $\text{level}(\omega) < \text{level}(\sigma)$ , then by (OneLevel)  $\text{level}(\sigma) = \text{level}(\omega) + 1$ , and (MaubachB) applies directly, giving  $i = \text{type}(\omega)$  – which is the “moreover” clause – and consequently  $i \notin \{0, \text{type}(\sigma)\}$  (since  $\text{type}(\cdot) \in \{1, \dots, m\}$  never takes the value 0, and  $\text{type}(\omega) \neq \text{type}(\sigma)$  because  $\text{level}(\omega) = \text{level}(\sigma) - 1$  shifts level mod  $m$  by exactly one). This proves the second case, again by contraposition.  $\square$

**Remark.** The PDF text extracted from the thesis’s source loses the two comparison glyphs in the lemma’s statement entirely (a font-encoding artifact of the document, not a content error) – both lines print as bare  $\text{level}(\omega) \quad \text{level}(\sigma)$  with the relation missing. We reconstructed the correct direction independently from the two proof paragraphs above (the first paragraph derives  $\text{level}(\omega) > \text{level}(\sigma) \Rightarrow i \in \{0, \text{type}(\sigma)\}$ , whose contrapositive gives the first printed case; the second derives  $\text{level}(\omega) < \text{level}(\sigma) \Rightarrow i \notin \{0, \text{type}(\sigma)\}$ , giving the second), then confirmed it against the original page image (page 48 of the PDF), which does show  $\leq$  then  $\geq$  exactly as above. Worth flagging explicitly: this direction is the opposite of the special/generic pairing in Mitchell’s analogous Lemma 1 ( $i = 0 \Rightarrow \text{level}(\omega) \leq \text{level}(\sigma)$ ,  $i \neq 0 \Rightarrow \text{level}(\omega) \geq \text{level}(\sigma)$  there) – not a typo, but a real consequence of (MaubachB)’s built-in asymmetry: unlike Mitchell’s fixed special index 0, Maubach’s special pair always references type of the shallower cell, regardless of which of  $\sigma, \omega$  that is.

### 2.3 Lemma 6: the child relation

**Lemma** (Lemma 6). *Let  $K$  be a mesh consisting of a single  $m$ -dimensional cell  $\omega$ , and  $K' = K(\omega[0, l], v)_l$  with  $l > 0$ . Then the cells  $c_0\omega, c_1\omega \in K'$  satisfy  $\partial_0 c_0\omega = \partial_{l-1} c_1\omega$ .*

*Proof.* Write  $\omega = \langle v_0, v_1, \dots, v_m \rangle$ . Applying  $(\omega[0, l], v)_l$  gives

$$c_0\omega = \langle v_0, \dots, v_{l-1}, v, v_{l+1}, \dots, v_m \rangle, \quad c_1\omega = \langle v_1, \dots, v_l, v, v_{l+1}, \dots, v_m \rangle,$$

and dropping position 0 from  $c_0\omega$  or position  $l-1$  from  $c_1\omega$  both yield  $\langle v_1, \dots, v_{l-1}, v, v_{l+1}, \dots, v_m \rangle$ .  $\square$

This is the geometric content of “splitting the edge  $[0, l]$ ”: the two children share the facet obtained by dropping the *old* endpoint on each side, at the position where the *new* vertex  $v$  was inserted on the other side – exactly the bookkeeping the LPT sibling-facet rule of Section 4 will later reuse verbatim, with zero floating-point work.

### 2.4 Lemma 7: propagation preserves the invariants

**Lemma** (Lemma 7). *Let  $K$  be a mesh consisting of two adjacent  $m$ -dimensional cells  $\sigma, \omega$  with  $\partial_i \sigma = \partial_j \omega$ , such that  $0 \leq \text{level}(\sigma) - \text{level}(\omega) \leq 1$ . Then, if  $K$  satisfies the Maubach invariants, so does  $K' = K(\omega[0, l], v)_l$  with  $l = \text{type}(\omega)$ .*

*Proof.* There are seven possible cases, distinguished by whether  $\sigma, \omega$  are at the same level or one apart, and by which of  $\{i, j\}$  equal 0 or  $l$ . In every case,  $\omega$ ’s subdivision produces  $c_0\omega, c_1\omega$  with  $\partial_0 c_0\omega = \partial_{l-1} c_1\omega$  (Lemma 6), which already satisfies (MaubachA) on its own; the seven cases only differ in *which* additional relation involving  $\sigma$  must be checked, and each resolves to a single facet identity that satisfies either (MaubachA) or (MaubachB) directly:

1.  $\text{level}(\sigma) = \text{level}(\omega)$ ,  $i = j = 0$ :  $\partial_0 \sigma = \partial_l c_1\omega$ , satisfying (MaubachB).
2.  $\text{level}(\sigma) = \text{level}(\omega)$ ,  $i = j = l$ :  $\partial_l \sigma = \partial_l c_0\omega$ , satisfying (MaubachB).

3.  $\text{level}(\sigma) = \text{level}(\omega)$ ,  $i = j = k \notin \{0, l\}$ :  $\sigma$  is also subdivided (into  $c_0\sigma, c_1\sigma$ , with the same Lemma-6 relation), and  $\partial_k c_0\sigma = \partial_k c_0\omega$ ,  $\partial_{k-1} c_1\sigma = \partial_{k-1} c_1\omega$ , both satisfying (MaubachA).
4.  $\text{level}(\sigma) = \text{level}(\omega)$ ,  $i = 0, j = l$ :  $\partial_0\sigma = \partial_l c_0\omega$ , satisfying (MaubachB).
5.  $\text{level}(\sigma) = \text{level}(\omega)$ ,  $i = l, j = 0$ :  $\partial_l\sigma = \partial_l c_1\omega$ , satisfying (MaubachB).
6.  $\text{level}(\sigma) = \text{level}(\omega) + 1$ ,  $i = l, j = 0$ :  $\partial_l\sigma = \partial_l c_1\omega$ , satisfying (MaubachA).
7.  $\text{level}(\sigma) = \text{level}(\omega) + 1$ ,  $i = l, j = l$ :  $\partial_l\sigma = \partial_l c_0\omega$ , satisfying (MaubachA).

In every case  $K'$  satisfies the Maubach invariants.  $\square$

**Remark.** *Each of the seven cases was checked directly against Lemma 6's own two identities and Definition 9's two clauses; none requires anything beyond substituting the child cell's known facet identity into whichever of (MaubachA)/(MaubachB) the resulting level/index pair falls under. Cases 1–5 (same level) all end up invoking (MaubachB) precisely because  $\sigma$ , unmoved, is now one level shallower than  $\omega$ 's new children; cases 6–7 invoke (MaubachA) because  $\sigma$  was already one level deeper than  $\omega$  and stays so relative to  $\omega$ 's children. This is a satisfying cross-check: it is exactly the pattern Lemma 5 predicts for which relation must hold once the levels involved are known.*

**Theorem** (Theorem 8). *Let  $K$  be an  $m$ -dimensional mesh satisfying the Maubach invariants and  $\sigma \in K$ . Then the mesh  $K'$  resulting from Maubach's subdivision algorithm applied to  $(K, \sigma)$  also satisfies the Maubach invariants, and the algorithm is consistent, alternating, and balanced.*

The proof is by induction on  $\text{level}(\sigma)$ , exactly mirroring the Mitchell case: at the minimal level, Lemma 5 shows every cell sharing  $\sigma$ 's subdivision edge  $\epsilon = \sigma[0, \text{type}(\sigma)]$  is also at that level, so the algorithm's own recursive step is never triggered; Lemma 7, applied to every pair of adjacent cells in  $\text{st}(\epsilon, K)$ , then shows the resulting mesh still satisfies the invariants. At any higher level, Lemma 5 and the induction hypothesis reduce to the same base case one level down. The one detail specific to Maubach (absent from Mitchell's proof) is that  $K$  must already be conforming, since the algorithm implicitly walks the star of the subdivision edge.

## 2.5 Two constructive routes, and their common mechanism

The thesis exhibits two families of meshes satisfying Definition 9 outright.

**Theorem** (Theorem 9). *Let  $\text{Tr}(\Sigma)$  be the canonical triangulation of a product of  $n$  simplices  $\Sigma = \Delta^{m_1} \times \dots \times \Delta^{m_n}$ . Then for any  $\sigma, \omega \in \text{Tr}(\Sigma)$ ,  $\partial_i\sigma = \partial_j\omega \Rightarrow i = j$ ; in particular  $\text{Tr}(\Sigma)$  satisfies the Maubach invariants.*

**Theorem** (Theorem 10). *Let  $K$  be any  $n$ -dimensional mesh. Then for any  $\sigma, \omega \in B(K)$  (its barycentric subdivision),  $\partial_i\sigma = \partial_j\omega \Rightarrow i = j$ ; in particular  $B(K)$  satisfies the Maubach invariants.*

Both proofs share one mechanism, worth isolating explicitly because it is exactly what fails for the mesh this report actually uses. In Theorem 9, every cell's  $j$ -th vertex is  $v_L$  with  $|L| = j$ , where  $|L|$  is the *lattice rank* of the multi-index  $L \in \{0, \dots, m_1\} \times \dots \times \{0, \dots, m_n\}$  – a quantity defined once, globally, on the whole vertex set, independent of which cell we look at. In Theorem 10, the analogous role is played by  $d(\delta)$ , the dimension of the flag element  $\delta$  labelling a barycentric-subdivision vertex. In both cases, a shared facet's missing local index is forced to equal the (unique) rank value absent from that facet's vertex ranks – so it is automatically the *same* value on both sides. Call this a *global flag/rank* structure: a function  $\text{rank} : V \rightarrow \{0, \dots, m\}$  on the vertex set of the entire complex such that every top cell's vertices realize every rank exactly once, with each cell's local vertex order taken to be its rank. Whenever such a function exists, Maubach compatibility (indeed the strictly stronger  $i = j$  always) follows for free.

Kuhn’s own triangulation of a cube is the  $n = m$ ,  $m_1 = \dots = m_n = 1$  case of Theorem 9, and it is this special case – not the general product construction – that the rest of the thesis, and the LPT machinery of Section 4, was originally built around. Neither theorem, however, says anything about a mesh that is *neither* a product of simplices *nor* a barycentric subdivision – which is exactly the situation for the triangulation used in this report. (A related, more elementary question – whether the SAME monotone-path mechanism still works when each *factor* of the product is itself an already-compatible mesh with more than one cell, rather than a bare simplex – is answered in Section A.) Since the thesis was published only in Portuguese, Section B also gives literal English translations of Theorems 8, 9 and 10’s own proofs, together with Maubach’s subdivision algorithm itself.

### 3 Compatibility beyond the cube: a combinatorial search

#### 3.1 Definition 9 as a coloring problem

The global flag/rank mechanism of Section 2.5 is a *sufficient*, not necessary, route to Maubach compatibility – Definition 9 itself only constrains *pairs of adjacent cells*, and says nothing about vertices globally. This suggests a direct, local reformulation. Restricted to the cells of an initial *seed* mesh (before any subdivision, so  $\text{level} \equiv 0$  and hence  $\text{type}(\sigma) = m$  for every cell), (MaubachA) reads

$$\partial_i \sigma = \partial_j \omega \implies i = j \text{ or } \{i, j\} = \{0, m\}.$$

This is a per-cell *local vertex ordering* problem: choose, for each  $m$ -cell, a bijection between its  $m + 1$  vertices and local positions  $\{0, \dots, m\}$  (equivalently, a coloring of the facet *opposite* each vertex by that vertex’s position) such that whenever two cells share a facet, the two colors assigned to it agree, up to the single allowed exception of the pair  $\{0, m\}$ . Phrased on the mesh’s *dual graph* (one node per cell, one edge per shared facet), this is a graph-coloring search: find a proper edge-coloring of the dual graph by  $m + 1$  colors that also respects, cell by cell, which color each cell may legally place on which of its own facets. Unlike Section 2.5’s constructions, this question is decidable for *any fixed* triangulation, independent of whether it carries any algebraic symmetry, simply by search.

#### 3.2 Gaifullin’s triangulation

The mesh used throughout the rest of this report is A. Gaifullin’s 15-vertex triangulation of  $\mathbb{CP}^2$  [7] (Construction 1.1 and §3 of that paper), independently cross-checked against GAP’s [9] `simpcomp` [10] library (SCLib entry #397, “Gaifullin  $\mathbb{CP}^2$ ”, identical  $f$ -vector and facet list up to relabeling). Its 15 vertices are labelled by  $(V_4 \setminus \{e\}) \cup (\{1, 2, 3, 4\} \times \{1, 2, 3\})$ , where  $V_4$  is the Klein four-group inside  $S_4$ ; its 108 four-simplices have  $f$ -vector  $(f_0, \dots, f_4) = (15, 90, 240, 270, 108)$ , matching  $\mathbb{CP}^2$ ’s own Euler characteristic 3 ( $f_0 - f_1 + f_2 - f_3 + f_4 = 15 - 90 + 240 - 270 + 108 = 3$ ).

Two combinatorial facts about this triangulation matter here, both statements about its 108-node, 5-regular dual graph (one node per cell, one edge per shared tetrahedral facet –  $108 \times 5/2 = 270$  edges, matching  $f_3 = 270$  exactly, with a facet-owner histogram of  $\{2 : 270\}$  confirming every facet is shared by exactly two cells, a genuine closed pseudomanifold):

- **Chess colouring (Gaifullin’s own term) / orientation coherence.** Gaifullin’s paper [7] proves this triangulation admits a *chess colouring*: a black/white 2-colouring of the 108 four-simplices such that any two sharing a facet get different colours – exactly a proper 2-colouring of the dual graph, i.e. the dual graph is *bipartite*. This is not an incidental fact this project discovered independently; it is a defining property of Gaifullin’s construction (his own theorem states this triangulation is the minimal one admitting such a colouring), which we simply re-verified directly (breadth-first 2-colouring, 0 conflicts, cross-checked against `simpcomp` [10]). It is exactly the condition needed for a globally coherent choice

of cell orientations under the standard  $(-1)^{\text{local index}}$  boundary-operator sign convention: adjacent cells must always induce *opposite* orientations on their shared facet, and a sign propagation over a non-bipartite dual graph would hit a contradiction on some odd cycle.

- **Maubach compatibility.** The same dual graph admits a proper 5-edge-colouring realizing  $\partial_i \sigma = \partial_j \omega \Rightarrow i = j$  for every one of the 270 shared facets (0 same-index mismatches, the *strong* form, with no need of the  $\{0, m\}$  exception at all) – found by exact backtracking search over the dual graph’s edges.

**Remark** (Why the second fact was never really in doubt). *These are not two independent coincidences. By König’s edge-colouring theorem, every bipartite graph has chromatic index exactly equal to its maximum degree  $\Delta$  (bipartite graphs are always “Class 1”). The dual graph is 5-regular ( $\Delta = 5$ , since every 4-simplex has exactly 5 facets); once it is known to be bipartite – Gaifullin’s own chess colouring, above – König’s theorem guarantees a priori that a proper 5-edge-colouring exists. So the backtracking search for Maubach compatibility was not searching for something that might not exist: its existence already followed from Gaifullin’s own chess-colouring theorem: the search only had to construct an explicit instance, which the implementation then uses directly as the 108 cells’ local vertex order. The implication runs one way only, though: bipartiteness is the strictly stronger property. A 5-regular graph can be Class 1 (5-edge-colorable) without being bipartite – which is exactly Kühnel’s own situation in Section 3.3 below: Maubach-compatible, but not orientable.*

The 108 cells’ local vertex orders, as actually used everywhere in the implementation, are precisely the ones this search assigned: local position  $i$  of a vertex in a cell is the color the 5-edge-coloring gave to the facet opposite it.

### 3.3 A cautionary counterexample: Kühnel’s triangulation

An earlier stage of this project attempted the same route with W. Kühnel and T. Banchoff’s classical 9-vertex, 36-cell minimal triangulation of  $\mathbb{CP}^2$  instead (reproduced in R. Schwartz’s [8]). An exact backtracking search there *does* find a valid 5-edge-coloring of its own 36-node dual graph realizing  $i = j$  always – so Kühnel’s raw complex is Maubach-compatible in exactly the same strong sense as Gaifullin’s. Its dual graph, however, is *not* bipartite, so global orientation coherence fails outright: no per-cell sign choice makes every adjacent pair induce opposite orientations. The workaround adopted at the time – barycentric subdivision, which by Theorem 10 is *guaranteed* to be both Maubach-compatible and orientable, since its global flag/rank structure fixes both properties at once – introduced its own, unrelated numerical defect: three of Kühnel’s nine original points are exact affine combinations of two others, so three of the barycentric subdivision’s new flag-centroid vertices collapse exactly onto already-existing vertices, producing degenerate, zero-length-edge cells that Maubach’s own longest-edge rule can never split. Gaifullin’s triangulation was adopted instead precisely because it satisfies *both* combinatorial properties directly, with no subdivision step and hence no vertex-coincidence risk at all.

## 4 An economical mesh representation: LPT codes and neighbor finding

### 4.1 Atalay & Mount’s pointerless codes

A naive implementation of Maubach subdivision keeps every cell ever created – bisected or not – in an append-only container, since other cells’ incidence data may reference it; memory then grows monotonically with refinement depth. This project’s first  $\mathbb{CP}^2$  implementation, built on

the library’s general-purpose `nmt<4>` semi-simplicial container, measured this directly: roughly 4–5 GB peak resident memory around subdivision depth 18 on a machine with 7.2 GB of RAM.

Atalay & Mount’s *LPT code* [6] avoids this entirely, for the specific case of Kuhn’s canonical triangulation of a hypercube: a cell is identified by a small, self-contained integer – no pointers, no stored incidence – from which both the cell’s own geometry (their Theorem 1) and its same-*orthant* neighbors’ codes (their Theorem 2) are computable in  $O(1)$ -ish time by table lookup alone. The key structural fact their scheme exploits is that Kuhn’s  $m!$  root simplices are not  $m!$  independent objects, but a single orbit of  $\text{Sym}(m)$  acting on one reference simplex; composing a root’s signed permutation with the elementary swap that crosses a same-orthant facet always lands on *another member of that same orbit*, by group closure – so a same-orthant neighbor never needs to consult anything beyond the permutation-table itself.

## 4.2 glpt: extending LPT to a non-orbit seed

This project’s `glpt` code (`~/code/lpt/glpt.hpp`) reuses Atalay & Mount’s (`level`, `sigperm`, `orthant_level`) machinery verbatim – it is dimension-generic and independent of which seed cell a code descends from – and adds one field, `seed_index`  $\in \{0, \dots, 107\}$ , selecting which of Gaifullin’s 108 root cells a given code belongs to. The resulting cell identity is a handful of bit-packed fields, all fitting in one 64-bit machine word (`uint64_t`):

| field        | seed   | level | sigperm | orthant_level | orth[0..3]         | total     |
|--------------|--------|-------|---------|---------------|--------------------|-----------|
| width (bits) | 7      | 2     | 9       | 4             | $4 \times 10 = 40$ | <b>62</b> |
| range        | 0..107 | 0..3  | 0..383  | –             | –                  | (of 64)   |

– 62 of the word’s 64 bits, no pointers, no per-cell incidence data stored anywhere. (`orth` is not a single field: it is one 10-bit coordinate *per dimension* –  $\text{dim} = 4$  coordinates, since one bit is appended to each of them on every subdivision round – not the single field a first glance at the table might suggest.) Storage cost is  $O(1)$  per *current leaf*, not per cell ever created (Section 4.4 below).

Gaifullin’s 108 cells are **not** a group orbit – they are 108 independently-specified vertex-order lists, related to each other only by which facets happen to coincide, a fact established by the search of Section 3, not by any algebraic symmetry. So the orbit-closure property Atalay & Mount’s same-orthant formula relies on simply does not hold here: empirically, a same-orthant ( $1 \leq i \leq \text{dim}$ ) facet crossing changes `seed_index` in roughly a quarter of sampled cases at realistic depths, confirmed both statistically and by hand for a concrete case (a facet whose barycentric weight, relative to the seed’s own five vertices, is exactly zero once the excluded vertex is dropped – a genuine boundary touch). A first attempt at reusing the paper’s formula unchanged was cross-checked against the invariant that every cell’s vertices must be convex combinations of its seed’s own five original points (since the whole tree is built by repeated midpointing): it produced a barycentric weight outside  $[0, 1]$  – a geometric impossibility – in 23.0% of sampled same-orthant cases, lining up exactly with the  $\sim 24\%$  of cases that legitimately do cross a seed boundary.

## 4.3 The neighbor function

`glpt::neighbor(i, &result)` resolves a query facet  $i$  of a cell by one of four different rules, chosen by an exact combinatorial gate – this is the one piece of the whole implementation genuinely reshaped by moving off Kuhn’s cube, and is worth describing in full:

1. **The sibling facet** (the one introduced by the cell’s own most recent bisection:  $i = \text{dim}$  for a 0-child,  $i = \text{type}$  for a 1-child) is resolved with *no floating-point arithmetic at all*, purely combinatorially, as `parent().child(other bit)`. By Lemma 6’s own identity

( $\partial_0 c_0 \omega = \partial_{l-1} c_1 \omega$ ), this is *always* exactly the correct neighbor, same seed, no ambiguity – routing it through any geometric path instead was tried first and broke 5–15% of sampled cases, since that facet sits at a finer scale than a generic geometric-nudge tolerance is tuned for.

2. For every other facet, an *exact* (integer, dyadic-rational, no floating point) test, `root_boundary_facet()`, decides whether the facet lies precisely on one of the seed’s own five bounding hyperplanes – computed from the same barycentric weights Atalay & Mount’s Theorem 1 gives, but kept as exact integers, since repeated bisection is exact in binary and the denominator never exceeds  $2^{\text{MAX\_ORTHANT\_LEVEL}+1}$ .
3. If that test says *interior* (no seed boundary), the original  $O(1)$  formula (their Theorem 2) is trusted – verified, by direct comparison against an independently-implemented geometric fallback, to agree in 132799/132801 (99.9985%) of sampled cases.
4. If the test says *boundary* (always true for a root cell, every one of whose 5 facets is a boundary by definition), the crossing is resolved through Gaifullin’s own precomputed  $108 \times 5$  adjacency table, built from the same “identical local order on both sides” fact Section 3’s search established – landing on a fresh root of the new seed, with no permutation correction needed.

A genuinely unresolved case, `GLPT_DEEP_CROSSING_UNIMPLEMENTED`, is reported explicitly (never guessed) when the region across a facet has already been refined *deeper* than the querying cell – enumerating which of possibly several finer descendant leaves border a given ancestor-level facet was judged out of scope. Orientation coherence at arbitrary depth (not just at the 108 root cells) was checked directly: 750 000 sampled (cell, facet) pairs across the full representable depth range, 0 violations.

#### 4.4 The current-leaf set: `glpt_tree`

A single packed code identifies a cell, but the *mesh* is the set of codes that are currently leaves. `glpt_tree` stores exactly that set, as an open-addressing hash table (linear probing, one 64-bit slot per entry, backward-shift deletion since bisection’s continuous erase-two-insert churn would otherwise accumulate tombstones indefinitely) – table occupancy is always exactly the current leaf count, never a historical operation count, unlike the append-only `nmt<4>` container it replaces.

Two operations complete the picture. `compat_bisect(r)` bisects  $r$ , first recursively forcing whichever neighbors are needed to keep the mesh *graded* (no facet ever separates cells more than one level apart) – a direct, ported “needs a neighbor finer first” recursion, using exactly `glpt::neighbor()` and `exists()` the way an earlier, cube-only implementation of the same idea already did, with three real bugs found (not merely by inspection but by an exhaustive graded-invariant test) and fixed along the way: a same-level neighbor that turns out to be an already-bisected root has no parent to recurse into; a neighbor whose parent is already mid-recursion higher up the same call stack must not be processed twice; and the absence of a same-level neighbor does not always mean it is coarser – it can instead mean that cell was already refined *finer*, by the very same top-level call, moments earlier, through a different facet entirely. Verified after the fixes at  $\sim 40\,000$  `compat_bisect()` calls cascading to nearly 4 million leaves, 11.6 million facet checks, 0 graded-invariant violations. `neighbor_leaf(r, i, &result)` then layers the tree’s actual state on top of the bare combinatorial `neighbor()`: it walks up the ancestor chain when the combinatorial candidate is not itself a current leaf (always resolvable, since a cell has exactly one ancestor chain), and reports `REFINED_FURTHER` – safely skippable, since it means that region was already visited from elsewhere – when the candidate has already been split further than the query.

## 5 A continuation algorithm for Riemann-surface triangulation

### 5.1 Problem setup

A smooth plane curve  $F(X, Y, Z) = 0$  (homogeneous, degree  $n$ ) sits inside  $\mathbb{CP}^2$  with real codimension 2. Seeding the mesh with Gaifullin’s 108 cells and Maubach-bisecting adaptively, a per-cell extraction step (`extract_cell`) turns each visited leaf’s boundary root-crossings – located by Newton’s method on each of its 10 distinct triangular 2-faces, each face solved once and cached by its global vertex-id triple – into a graph on those crossing points, decomposed into closed polygonal cycles that approximate the curve inside that cell. A cell is called *bad* when that decomposition fails (a vertex of the crossing graph with degree other than 2, or an unhandled multiple-crossing configuration on a single facet) – typically a near-tangency or several close crossings on one facet.

### 5.2 From global refinement to continuation

The algorithm’s earlier refinement criterion, *gradient dispersion* ( $1 - \kappa$ , where  $\kappa$  measures how aligned  $\nabla F$  is across a cell’s own vertices), is evaluated identically whether or not a cell is anywhere near the curve, and drives a priority queue seeded from all 108 roots. This refines the entire ambient  $\mathbb{CP}^2$  – a real 4-manifold – to resolve a curve of real dimension 2, and is correspondingly wasteful.

The final version of the algorithm instead uses *continuation*: starting from one cell the curve is confirmed to touch, it explores outward through only the cells the curve actually reaches, using precisely the two combinatorial relations of Section 4 – `compat_bisect/recent_leaves` to go deeper, and `neighbor_leaf` to move sideways once a cell has reached the requested depth.

### 5.3 The algorithm

---

**Algorithm 1** Continuation-based Riemann-surface triangulation (default Phase 1 of `riemann_cp2_glpt.cpp`)

---

**Require:** Gaifullin’s 108-cell seed mesh  $T$  (all roots present), curve  $F$ , target depth  $D$

- 1:  $s \leftarrow$  first  $s \in \{0, \dots, 107\}$  with `any_edge(extract_cell(s)) = true`  $\triangleright$  early-exit scan; skipped entirely for  $s' > s$
- 2:  $visited \leftarrow \emptyset$ ;  $frontier \leftarrow \{s\}$
- 3: **while**  $frontier \neq \emptyset$  **do**
- 4:   pop  $c$  from  $frontier$
- 5:   **if**  $c \notin T$  **or**  $c \in visited$  **then continue**  $\triangleright$  superseded by an earlier cascade
- 6:   **end if**
- 7:    $visited \leftarrow visited \cup \{c\}$
- 8:    $r \leftarrow \text{extract\_cell}(c)$
- 9:   **if not**  $r.any\_edge$  **then continue**  $\triangleright$  curve doesn’t reach here
- 10:   **end if**
- 11:   **if** `simplex_level(c) < D` **then**
- 12:      $T.compat\_bisect(c)$
- 13:      $frontier \leftarrow frontier \cup T.recent\_leaves()$   $\triangleright$  includes any cell a conformity cascade touched elsewhere
- 14:   **else**
- 15:     record  $c$  as a final touched leaf (using  $r$  already computed)
- 16:     **for**  $i = 0, \dots, \text{dim}$  **do**
- 17:       **if**  $T.neighbor\_leaf(c, i) = (\text{FOUND}, n)$  **and**  $n \notin visited$  **then**
- 18:          $frontier \leftarrow frontier \cup \{n\}$
- 19:       **end if**
- 20:     **end for**
- 21:   **end if**
- 22: **end while**

---

The face-crossing cache built while computing `any_edge` above is shared with the subsequent per-polygon extraction/output pass, so no 2-face crossing is ever solved twice.

#### Correctness sketch

If the curve crosses cell  $A$ ’s facet  $f$ , shared with a neighbor  $B$ , it must be present, at least infinitesimally, on  $B$ ’s side of  $f$  too – a 1-dimensional curve entering a 4-cell through a shared 3-facet cannot stop exactly at that facet – so `any_edge(B)` is guaranteed once the search reaches it via a genuine crossing facet, and the propagation never has to guess. This is, however, an argument for recovering a single *connected component* of  $\{F = 0\}$ : a reducible curve with several irreducible factors would need one seed per component, which the current implementation does not attempt.

### 5.4 Measured results

For every one of the six catalog test curves (conic, Fermat cubic, a generic Hesse-pencil cubic, a smooth quartic, a parabola, and a smooth elliptic curve – all connected), continuation recovers the exact same final leaf set, and the exact same bad-cell count, as an independent reference that applies the same `any_edge` test to all 108 roots directly rather than propagating – checked at depths 8, 12, and 14. Two further, independent measurements:

- **Mesh size**, relative to the earlier gradient-dispersion Phase 1 at the same depth and refinement threshold: 62–82% of the old leaf count at depth 8, falling to 17–34% at depth 14 – a 3–6 $\times$  smaller mesh, with the gap widening as depth increases.
- **Bad-cell rate**: never worse than the old criterion across the six curves, and strictly better on two of them (a conic, 20.13%  $\rightarrow$  17.81%; a parabola, 18.78%  $\rightarrow$  17.33%) – exactly the curves where the old gradient-dispersion signal had been stopping refinement of a touched cell before it reached full depth.

A concrete run, the deepest attempted so far: the smooth elliptic curve  $X^2Z - Y^3 + YZ^2 = 0$ , seed rotated by a random unitary change of basis (to avoid the curve resonating with the seed’s own coordinate axes), at depth 20, produced 4 620 102 leaves, with 222 981 cleanly-extracted cells and 1 348 bad ones (a 0.60% bad-cell rate, itself continuing the already-documented trend that this rate falls with depth). Two additional targeted repair rounds – re-bisecting specifically the cells that failed extraction, up to a small extra depth budget, then re-extracting – brought this down to 678 bad cells out of 227 355 (0.30%).

The bad-cell rate measures *topological* correctness – whether a cell’s crossing graph decomposes into simple closed polygons at all – but says nothing about how accurately those polygons’ own vertices actually sit on the curve. To check that separately, we evaluated  $|F(v)|$  at every *distinct* extracted crossing point  $v$  (deduped by output- vertex identity, the same identity Section 5’s alpha projection uses for its own global vertex dedup) for this same depth-20 run: 164 390 distinct points – matching, as a cross-check, the vertex count `-alpha-projection` itself reports for this run – with  $|F(v)|$  median  $1.0 \times 10^{-16}$ , mean  $3.0 \times 10^{-14}$  (pulled up by a small tail), 95th/99th percentile  $2.0 \times 10^{-13}/4.3 \times 10^{-13}$ , and maximum  $9.8 \times 10^{-13}$ . Every one of these values is within a few orders of magnitude of double-precision machine epsilon ( $\approx 2.2 \times 10^{-16}$ ) relative to  $F$ ’s own natural scale on unit- norm homogeneous coordinates – Newton’s method (Section 5’s own root-finding, inside `extract_cell`) is not merely finding *a* crossing point per facet, but converging to the true zero locus to essentially full floating-point precision.

## 5.5 Limitations

The algorithm as implemented recovers exactly one connected component of the curve; a reducible curve needs one seed per irreducible factor, not yet implemented. A separate investigation, summarized here only for completeness, ruled out one plausible cause of the residual bad cells: the geodesic (Fubini–Study) midpoint used for every edge bisection does *not* split the two children’s volumes evenly, unlike the Euclidean case – measured directly via a Cayley–Menger volume proxy built from true geodesic distances – but that imbalance turns out *not* to correlate with which cells end up bad; if anything, the bad-cell rate *falls* as the imbalance of the bisection that created a cell grows. Bad cells remain concentrated instead on genuine near-tangencies and multiple close crossings on a single facet – confirmed quantitatively, not just asserted, by directly instrumenting `extract_cell()` itself (all six catalog curves, depth 14: 136 786 touched cells, 8.02% bad):

- **Direct cause.** 81.6% of bad cells fail on a *degree* mismatch (some crossing-graph node ends up with degree  $\neq 2$ , even though every individual facet’s own local crossing count was already one of  $\{0, 1, 2, 4\}$ ); the remaining 18.4% fail on an outright *irregular* facet (ntn = 3: three distinct crossing points on a single tetrahedral facet’s four triangular 2-faces – always exactly 3 in this sample, never more, since `extract_cell`’s own buffer caps the count at 4 by construction). No cell failed purely at the final cycle-walk step once its facet counts and degrees were already consistent.
- **Single-facet touching.** A facet with ntn = 1 (the curve grazing it at one point, without a clean two-point crossing) is literally a local near-tangency. Cells with at least one such facet have a 29.9% bad rate, versus 4.1% for cells with none – a  $\sim 7\times$  difference.

- **Pairing ambiguity.** The clearest signal: for a facet crossed at  $\text{ntn} = 4$  of its sub-triangles, the extraction must choose which 2 of 3 candidate pairings connect into arcs, by minimum total geodesic length. Taking the ratio of the best pairing’s cost to the second-best’s (1.0 = perfectly tied) and averaging over every cell with at least one such facet: mean 0.42 (median 0.41) for cells that still decompose cleanly, versus mean 0.68 (median 0.80, 90th percentile 0.9999) for bad ones. The bad population sits almost exactly where the pairing choice becomes ambiguous – exactly the local signature of a facet close to tangent to the curve, where the four crossing points approach a symmetric configuration under which every pairing costs nearly the same.

These three independent checks agree, and together account for the residual bad-cell population left after continuation and `-repair-rounds`.

## 6 Conclusion

Maubach’s invariants (Definition 9, Chapter 4 of the thesis) are a purely combinatorial condition, and the thesis’s own two constructive routes to satisfying it – products of simplices and barycentric subdivision – both work by exhibiting a global vertex rank, a structure tied closely to Kuhn’s cube and to dimension  $\leq 3$ , the setting the rest of the thesis develops the idea in. This report showed the condition is more general than that: reformulated as a local, per-facet coloring problem on a mesh’s dual graph, it is decidable by direct search for *any* fixed triangulation, with no algebraic symmetry required at all, and Gaifullin’s 108-cell, real-dimension-4 triangulation of  $\mathbb{CP}^2$  satisfies it this way. Built on top of that mesh, an economical LPT-style cell code (extending Atalay & Mount’s original pointerless scheme, with its neighbor-finding rules substantially reworked once the seed cells stop forming a single group orbit) supports, as a genuine downstream application, an adaptive triangulation algorithm for algebraic curves – Riemann surfaces – living directly inside  $\mathbb{CP}^2$ , whose current, continuation-based version recovers a curve’s mesh at a fraction of the earlier algorithm’s memory cost, at depths and mesh sizes the original `nmt<4>`-based implementation could not reach at all.

## A Products of already-compatible meshes

This appendix answers a question that arose naturally, not from the  $\mathbb{CP}^2$  work of the main text, but from an earlier stage of the same broader project: the first implementation of `examples/top/riemann`’s own Riemann-surface triangulator, which realizes a curve’s ambient space not as  $\mathbb{CP}^2$  but as  $S^2 \times S^2$  (two identified Riemann spheres), seeded the mesh by triangulating each  $S^2$  factor as an *octahedron* (the boundary of the cross-polytope: 6 vertices, 8 triangular faces) and combining the two via the classical Eilenberg–Zilber *shuffle* product – the same monotone-lattice-path recipe Theorem 9 uses for a product of simplices, but now with each factor already a small multi-cell mesh rather than a single simplex. Theorem 9, taken literally, does not cover this: an octahedron is not a simplex. This raises the natural question answered below – does the same construction, applied to two meshes that are *themselves* already known to satisfy the thesis’s strong compatibility property, still produce a compatible product?

**Proposition** (Product of compatible meshes). *Let  $K_1$  be an  $m_1$ -dimensional mesh equipped with a global rank  $\text{rank}_1 : V(K_1) \rightarrow \{0, \dots, m_1\}$  such that every top cell  $\sigma_1 = \langle \sigma_1[0], \dots, \sigma_1[m_1] \rangle$  of  $K_1$  satisfies  $\text{rank}_1(\sigma_1[l]) = l$  for every  $l$  – i.e.,  $K_1$  satisfies the strong form  $\partial_i \sigma = \partial_j \omega \Rightarrow i = j$  of the Maubach invariants, realized (as in Theorems 9 and 10, Section 2.5) by an explicit global rank. Let  $K_2, \text{rank}_2, m_2$  be likewise. Let  $K_1 \times K_2$  be triangulated by the canonical cell-by-cell extension of the monotone-path construction: for every pair of top cells  $(\sigma_1, \sigma_2) \in K_1 \times K_2$ , triangulate the prism  $\sigma_1 \times \sigma_2$  exactly as in Theorem 9’s proof (vertices  $v_L = (\sigma_1[L^1], \sigma_2[L^2])$ ) for*

$L = (L^1, L^2)$  in the lattice  $P = \{0, \dots, m_1\} \times \{0, \dots, m_2\}$ , cells indexed by monotone paths in  $P$ ), and glue these triangulated prisms together. Then  $K_1 \times K_2$  also satisfies the strong Maubach invariants.

*Proof.* Define, on the vertex set  $V(K_1) \times V(K_2)$  of the product,

$$\text{RANK}(a, b) := \text{rank}_1(a) + \text{rank}_2(b),$$

which is well defined globally – it depends only on the actual vertex pair  $(a, b)$ , not on which prism  $\sigma_1 \times \sigma_2$  we view it through. Consider any top cell  $\tau = \langle v_{L_0}, \dots, v_{L_s} \rangle$  of the product triangulation ( $s = m_1 + m_2$ ), living in some prism  $\sigma_1 \times \sigma_2$ , with  $L_0, \dots, L_s$  a monotone path in  $P$  from  $(0, 0)$  to  $(m_1, m_2)$ . Since each step of a monotone path increases  $L^1 + L^2$  by exactly 1 and  $L_0 = (0, 0)$ , we have  $|L_l| := L_l^1 + L_l^2 = l$  for every  $l$ . Hence

$$\text{RANK}(\tau[l]) = \text{rank}_1(\sigma_1[L_l^1]) + \text{rank}_2(\sigma_2[L_l^2]) = L_l^1 + L_l^2 = l,$$

using  $\text{rank}_1(\sigma_1[L_l^1]) = L_l^1$  and  $\text{rank}_2(\sigma_2[L_l^2]) = L_l^2$ , which hold by hypothesis on  $K_1, K_2$  individually. So *every* top cell of the glued product – in any prism whatsoever – has its  $l$ -th vertex at global rank exactly  $l$ ; this did not use anything about  $\tau, \tau'$  being in the same or different prisms.

Now let  $\tau, \tau'$  be any two facet-adjacent cells of  $K_1 \times K_2$ , with  $\partial_i \tau = \partial_j \tau'$ . The  $s$  vertices of this shared facet have, read off from  $\tau$ 's own labelling, ranks  $\{0, \dots, s\} \setminus \{i\}$ , and read off from  $\tau'$ 's, ranks  $\{0, \dots, s\} \setminus \{j\}$  – but RANK is a single global function of the vertex, so these are the same set of ranks for the same set of vertices, forcing  $\{0, \dots, s\} \setminus \{i\} = \{0, \dots, s\} \setminus \{j\}$ , hence  $i = j$ .  $\square$

**Remark.** *This is exactly Theorem 9's own argument, with  $|L|$  replaced by RANK – Theorem 9 is recovered verbatim as the special case where  $K_1, \dots, K_n$  are each a single bare simplex  $\Delta^{m_i}$  (whose “rank” is simply its own vertex order, with nothing to verify, since a single simplex has no facet-adjacency at all to check). The genuinely new content of Section A is that it also handles  $K_1$  or  $K_2$  having more than one cell, which requires checking that gluing across different  $(\sigma_1, \sigma_2)$  prisms stays consistent – and the proof shows this is automatic, precisely because RANK was defined once, on the vertex set, rather than separately per prism.*

**Remark** (Verification on a small example). *Take  $K_1$  the path  $v_0 - v_1 - v_2$  (two edges), with  $\text{rank}_1(v_0) = \text{rank}_1(v_2) = 0$ ,  $\text{rank}_1(v_1) = 1$  (forcing the second edge to be oriented  $\langle v_2, v_1 \rangle$ , not  $\langle v_1, v_2 \rangle$ , to respect this rank), and  $K_2$  a single edge  $\langle w_0, w_1 \rangle$ . This is the smallest instance where one factor has more than one cell.  $K_1 \times K_2$  is two unit squares sharing an edge, each split into two triangles by the usual recipe. Checking all three non-trivial adjacent pairs by hand – the two triangles within one square, and the two triangles facing each other across the shared square-to-square edge – confirms  $i = j$  in every case, matching RANK exactly as Section A predicts.*

**Remark** (Local compatibility is not enough – a caveat). *Section A's hypothesis is that  $K_1, K_2$  each admit a global rank – which is stronger than merely satisfying the strong invariant pairwise, cell by adjacent cell (the sense in which Section 3's search establishes compatibility for Gaifullin's own triangulation, with no global rank claimed or checked there). The two notions are not automatically equivalent: consider the 1-dimensional mesh consisting of a single 3-cycle of edges (a triangle, as a graph, not as a filled 2-simplex). Every pair of adjacent edges can be locally reordered to agree at their shared vertex ( $i = j$ ), exactly as in the path example above – but no consistent global 2-coloring of its three vertices exists, since a 3-cycle is not bipartite. This is the same odd-cycle obstruction already met in Section 3 for Kühnel's triangulation, there blocking global orientation coherence rather than a rank; here it blocks the rank itself. So Section A may not be invoked for a product whose factors are only known compatible in the weaker, local sense – one must exhibit (or separately verify) an actual global rank on each factor first, exactly as the octahedron example below does.*

**Remark** (The motivating case: the octahedron). *An octahedron – vertices  $\pm e_1, \pm e_2, \pm e_3$ , one face per sign choice  $(\pm, \pm, \pm)$  – admits an explicit global rank realizing the strong invariant directly: set  $\text{rank}(\pm e_k) := k - 1 \in \{0, 1, 2\}$  (color a vertex by which coordinate axis it lies on, ignoring sign). Every face has exactly one vertex on each axis by construction, so every face is automatically rainbow under this coloring. Section A therefore applies directly to  $K_1 = K_2 = (\text{octahedron})$ , confirming that the  $S^2 \times S^2$  seed mesh’s own compatibility – observed, but not previously derived from a general statement – is an instance of a genuine theorem, not a coincidence particular to that one mesh.*

## B Translated proofs of Theorems 8–10, and Maubach’s algorithm

The thesis this report is based on [1] was published only in Portuguese. For direct traceability against the original source, this appendix gives literal English translations of the proofs of Theorems 8, 9 and 10 (stated without proof in Sections 2 and 2.5 of the main text), together with a full reproduction of Maubach’s own subdivision algorithm (*Listagem 4.2* of the thesis, p. 47).

### Theorem 8

**Theorem** (Theorem 8). *Let  $K$  be an  $m$ -dimensional mesh satisfying the Maubach invariants and  $\sigma$  a cell of  $K$ . Then the mesh  $K'$  resulting from applying Algorithm 4.2 to  $(K, \sigma)$  also satisfies the Maubach invariants. Moreover, the algorithm is consistent, alternating, and balanced.*

*Proof.* By induction on the level of  $\sigma$ . Let  $l$  be the smallest value assumed by the function `level` on  $K$ . If  $\text{level}(\sigma) = l$ , it follows from Lemma 5 that cells adjacent to  $\sigma$  that share the subdivision edge  $\epsilon = \sigma[0, \text{type}(\sigma)]$  also have level  $l$ . Hence the recursion on line 19 [of Algorithm 4.2] is not executed, and the loop between lines 8–26 does nothing more than traverse the star of  $\epsilon$  and store it in the list `st`. Applying Lemma 7 to every pair of adjacent cells  $(\sigma', \omega')$ , with  $\omega' \in \text{st}(\epsilon, K)$ , we get that the subdivision performed on line 28 results in a mesh  $K'$  satisfying the statement. If  $\text{level}(\sigma) > l$ , we can use Lemma 5 and the induction hypothesis on line 19 to reduce to a case similar to the initial one.  $\square$

### Theorem 9

**Theorem** (Theorem 9). *Let  $\text{Tr}(\Sigma)$  be a mesh representing the canonical triangulation of the product of  $n$  simplices  $\Sigma = \Delta^{m_1} \times \Delta^{m_2} \times \dots \times \Delta^{m_n}$ . Then, for any cells  $\sigma, \omega \in \text{Tr}(\Sigma)$ ,*

$$\partial_i \sigma = \partial_j \omega \implies i = j.$$

*In particular,  $\text{Tr}(\Sigma)$  satisfies the Maubach invariants.*

*Proof.* A basic property of the construction above is that for every cell  $\sigma \in \text{Tr}(\Sigma)$ ,

$$\sigma[j] = v_L \iff |L| = j.$$

Let  $\sigma, \omega \in \text{Tr}(\Sigma)$  be two adjacent cells, with  $\sigma = \langle v_{L_0}, \dots, v_{L_s} \rangle$  and  $\omega = \langle v_{L'_0}, \dots, v_{L'_s} \rangle$ , such that  $\partial_i \sigma = \langle v_{L''_0}, \dots, v_{L''_{s-1}} \rangle = \partial_j \omega$ . Then there is a unique  $k \in \{0, \dots, s\}$  such that  $k \notin \{|L''_0|, \dots, |L''_{s-1}|\}$ . Therefore  $|L_i| = k = |L'_j|$ , hence  $i = k = j$ . Note that  $k \notin \{0, s\}$ , since for every cell  $\sigma \in \text{Tr}(\Sigma)$ ,  $\sigma[0, s] = \langle v_{(0, \dots, 0)}, v_{(m_1, \dots, m_n)} \rangle$ .  $\square$

## Theorem 10

**Theorem** (Theorem 10). *Let  $K$  be an  $n$ -dimensional mesh. Then, for any cells  $\sigma, \omega \in B(K)$ ,*

$$\partial_i \sigma = \partial_j \omega \implies i = j.$$

*In particular,  $B(K)$  satisfies the Maubach invariants.*

*Proof.* A basic property of the construction above is that for every cell  $\sigma \in B(K)$ ,

$$\sigma[j] = v_\delta \iff d(\delta) = j.$$

Let  $\sigma, \omega \in B(K)$  be two adjacent cells, with  $\sigma = \langle v_{\delta_0}, \dots, v_{\delta_n} \rangle$  and  $\omega = \langle v_{\delta'_0}, \dots, v_{\delta'_n} \rangle$ , such that  $\partial_i \sigma = \langle v_{\delta''_0}, \dots, v_{\delta''_{n-1}} \rangle = \partial_j \omega$ . Then there is a unique  $k \in \{0, \dots, n\}$  such that  $k \notin \{d(\delta''_0), \dots, d(\delta''_{n-1})\}$ . Therefore  $d(\delta_i) = k = d(\delta'_j)$ , hence  $i = k = j$ .  $\square$

## Maubach's subdivision algorithm

Below is Maubach's subdivision algorithm (*Listagem 4.2* of the thesis, p.47), given here as pseudocode – translated in structure and notation, not transcribed literally from the (mildly archaic, templated-C++-style) original listing.  $k$  is exactly  $\text{type}(\sigma)$ ;  $\text{st}$  (built up as cells are visited) ends up being  $\text{st}(\epsilon, K)$ , the star of the subdivision edge  $\epsilon = \sigma[0, k]$  computed at the end; and the recursive call is precisely the “propagate to whichever neighbor has a smaller level first” step that Theorem 8's proof, above, inducts on.

---

**Algorithm 2** MAUBACHSUBDIVIDE( $K, \sigma$ ) (translated from *Listagem 4.2*, p.47)

---

**Require:** conforming  $m$ -dimensional mesh  $K$ , cell  $\sigma \in K$  to subdivide

```

1:  $k \leftarrow \text{type}(\sigma)$   $\triangleright = m - (\text{level}(\sigma) \bmod m)$ 
2:  $\text{st} \leftarrow$  empty list;  $\text{to\_visit} \leftarrow$  queue containing  $\sigma$ 
3: while  $\text{to\_visit}$  is not empty do
4:    $c \leftarrow \text{to\_visit.front}(); \text{to\_visit.pop}()$ 
5:   if  $c$  is not marked visited then
6:     mark  $c$  visited; append  $c$  to  $\text{st}$ 
7:     for  $i = 0, \dots, m$  do
8:       if  $i = 0$  or  $i = k$  then continue  $\triangleright$  these two facets pass to a child whole – see
          Lemma 7's own remark
9:     end if
10:     $c_a \leftarrow \text{adjacent}(c, i)$ 
11:    if  $c_a$  is not marked visited then
12:      if  $\text{level}(c_a) < \text{level}(c)$  then
13:        MAUBACHSUBDIVIDE( $K, c_a$ )
14:         $c_a \leftarrow \text{adjacent}(c, i)$   $\triangleright$  re-fetch: a child of  $c_a$  now occupies this facet
15:      end if
16:      push  $c_a$  onto  $\text{to\_visit}$ 
17:    end if
18:  end for
19: end if
20: end while
21:  $\epsilon \leftarrow \sigma[0, k]$   $\triangleright$  the subdivision edge
22: MAUBACHEDGESPLIT( $K, \epsilon, k, \text{st}$ )  $\triangleright$  bisects  $\epsilon$ , splitting every cell of  $\text{st} = \text{st}(\epsilon, K)$ 

```

---

## References

- [1] Vinícius Moreira Mello. *Novos Métodos Simpliciais em Computação Gráfica*. Tese de Doutorado, Instituto de Matemática Pura e Aplicada (IMPA), Rio de Janeiro, 2006. Advisor: Luiz Velho.
- [2] J.M. Maubach. *Local bisection refinement for  $N$ -simplicial grids generated by reflection*. SIAM J. Sci. Stat. Comput., 16:210–227, 1995.
- [3] W.F. Mitchell. *Optimal multilevel iterative methods for adaptive grids*. SIAM J. Sci. Stat. Comput., 13:146–167, 1992.
- [4] Jonas Gomes and Geovan Tavares. *Métodos Simpliciais em Computação Gráfica*. 17<sup>o</sup> Colóquio Brasileiro de Matemática, 1989.
- [5] W.B.R. Lickorish. *Simplicial moves on complexes and manifolds*. In Proceedings of the Kirbyfest, volume 2, pages 299–320, 1999.
- [6] F. Betul Atalay and David M. Mount. *Pointerless implementation of hierarchical simplicial meshes and efficient neighbor finding in arbitrary dimensions*. International Journal of Computational Geometry & Applications, 2007.
- [7] A.A. Gaifullin. *Construction of combinatorial manifolds with prescribed sets of links of vertices*. arXiv:0904.4222.
- [8] R.E. Schwartz. *Trisecting the 9-vertex complex projective plane*. arXiv:2205.00595.
- [9] The GAP Group. *GAP – Groups, Algorithms, and Programming*. <https://www.gap-system.org>.
- [10] Felix Effenberger and Jonathan Spreer. *simpcomp – a GAP toolbox for simplicial complexes*. ACM Communications in Computer Algebra, 44(4):186–189, 2010. arXiv:1004.1367.

## Note on AI assistance

The implementation this report describes – the `glpt/ glpt_tree` pointerless mesh, the Gaifullin-compatibility search, the neighbor-finding rules of Section 4, the continuation algorithm of Section 5, and the various diagnostic investigations (volume imbalance, bad-cell correlation) referenced along the way – was developed with Claude (Anthropic) acting as a coding assistant across many work sessions, under the author’s direction, review, and testing. This report itself was likewise drafted with Claude’s assistance, including the verification of Lemmas 5–7 in Section 2, the translations and pseudocode of Section B, and Section A’s proof – again under the author’s direction and review throughout.